

Experiment 'p_14_1_selection_sort_desc_seq' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_14_1_selection_sort_desc_seq

Natural language statement: null

Method signature: p_14_1_selection_sort_desc_seq(inputs: seq<int>) returns (sorted: seq<int>)

Ensures

- forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] >= sorted[j]
- multiset(inputs) == multiset(sorted)

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_14_1_selection_sort_desc_seq(inputs: seq<int>) returns (sorted: seq<int>)`

The method should respect the following contract:

ensures forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] >= sorted[j], ensures multiset
↪ (inputs) == multiset(sorted)

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnPaVrHXsMzQAYBxTsOHPEArSM3aa

Model: gpt-5.5-2026-04-23

Created at: 1780668803

Finish reason: stop

Usage: 308 tokens in, and 7362 tokens out

```
{
  sorted := [];
  var rest := inputs;

  while |rest| > 0
    invariant multiset(inputs) == multiset(sorted) + multiset(rest)
    invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] >= sorted[j]
    invariant forall i, j :: 0 <= i < |sorted| && 0 <= j < |rest| ==> sorted[i] >= rest[j]
    decreases |rest|
  {
    var oldSorted := sorted;
    var oldRest := rest;
    assert |oldRest| > 0;
    assert forall i, j :: 0 <= i < j < |oldSorted| ==> oldSorted[i] >= oldSorted[j];
    assert forall i, j :: 0 <= i < |oldSorted| && 0 <= j < |oldRest| ==> oldSorted[i] >=
      ↪ oldRest[j];

    var maxIdx := 0;
    var k := 1;
    while k < |oldRest|
```

```

invariant 1 <= k <= |oldRest|
invariant 0 <= maxIdx < k
invariant forall t :: 0 <= t < k ==> oldRest[maxIdx] >= oldRest[t]
decreases |oldRest| - k
{
  var kk := k;
  var prevMaxIdx := maxIdx;
  if oldRest[kk] > oldRest[maxIdx] {
    maxIdx := kk;
    forall t | 0 <= t < kk
      ensures oldRest[maxIdx] >= oldRest[t]
    {
      assert oldRest[prevMaxIdx] >= oldRest[t];
      assert oldRest[maxIdx] > oldRest[prevMaxIdx];
    }
    assert oldRest[maxIdx] >= oldRest[kk];
  } else {
    forall t | 0 <= t < kk
      ensures oldRest[maxIdx] >= oldRest[t]
    {
    }
    assert oldRest[maxIdx] >= oldRest[kk];
  }
  forall t | 0 <= t < kk + 1
    ensures oldRest[maxIdx] >= oldRest[t]
  {
    if t < kk {
      assert oldRest[maxIdx] >= oldRest[t];
    } else {
      assert t == kk;
      assert oldRest[maxIdx] >= oldRest[kk];
    }
  }
  k := kk + 1;
}

assert k == |oldRest|;
assert forall t :: 0 <= t < |oldRest| ==> oldRest[maxIdx] >= oldRest[t];

var x := oldRest[maxIdx];
var prefix := oldRest[..maxIdx];
var suffix := oldRest[maxIdx + 1..];
var newRest := prefix + suffix;

assert |prefix| == maxIdx;
assert |suffix| == |oldRest| - (maxIdx + 1);
assert oldRest == prefix + [x] + suffix;
assert newRest == prefix + suffix;
calc {
  multiset(oldRest);
}
==

```

```

    multiset(prefix + [x] + suffix);
==
    multiset(prefix + [x]) + multiset(suffix);
==
    multiset(prefix) + multiset([x]) + multiset(suffix);
==
    multiset(prefix) + multiset(suffix) + multiset([x]);
==
    multiset(prefix + suffix) + multiset([x]);
==
    multiset(newRest) + multiset([x]);
}

sorted := oldSorted + [x];
rest := newRest;

assert |sorted| == |oldSorted| + 1;
assert |rest| == |oldRest| - 1;

forall i, j | 0 <= i < j < |sorted|
  ensures sorted[i] >= sorted[j]
{
  if j < |oldSorted| {
    assert i < |oldSorted|;
    assert sorted[i] == oldSorted[i];
    assert sorted[j] == oldSorted[j];
    assert oldSorted[i] >= oldSorted[j];
  } else {
    assert j == |oldSorted|;
    assert i < |oldSorted|;
    assert sorted[i] == oldSorted[i];
    assert sorted[j] == x;
    assert oldSorted[i] >= oldRest[maxIdx];
  }
}

forall i, j | 0 <= i < |sorted| && 0 <= j < |rest|
  ensures sorted[i] >= rest[j]
{
  var oldIndex: int;
  if j < maxIdx {
    oldIndex := j;
    assert j < |prefix|;
    assert rest[j] == prefix[j];
    assert prefix[j] == oldRest[j];
    assert rest[j] == oldRest[oldIndex];
  } else {
    oldIndex := j + 1;
    assert maxIdx <= j;
    assert j + 1 < |oldRest|;
    assert 0 <= j - maxIdx;
  }
}

```

```

    assert j - maxIdx < |suffix|;
    assert rest[j] == suffix[j - maxIdx];
    assert suffix[j - maxIdx] == oldRest[maxIdx + 1 + (j - maxIdx)];
    assert maxIdx + 1 + (j - maxIdx) == j + 1;
    assert rest[j] == oldRest[oldIndex];
  }
  assert 0 <= oldIndex < |oldRest|;
  if i < |oldSorted| {
    assert sorted[i] == oldSorted[i];
    assert oldSorted[i] >= oldRest[oldIndex];
  } else {
    assert i == |oldSorted|;
    assert sorted[i] == x;
    assert x >= oldRest[oldIndex];
  }
}

assert multiset(sorted) == multiset(oldSorted) + multiset([x]);
calc {
  multiset(sorted) + multiset(rest);
==
  (multiset(oldSorted) + multiset([x])) + multiset(newRest);
==
  multiset(oldSorted) + (multiset(newRest) + multiset([x]));
==
  multiset(oldSorted) + multiset(oldRest);
==
  multiset(inputs);
}
assert multiset(inputs) == multiset(sorted) + multiset(rest);
}

assert rest == [];
assert multiset(rest) == multiset([]);
assert multiset(sorted) + multiset(rest) == multiset(sorted);
assert multiset(inputs) == multiset(sorted);
}

```

Final Program

```

method p_14_1_selection_sort_desc_seq(inputs: seq<int>) returns (sorted: seq<int>)
  ensures forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] >= sorted[j]
  ensures multiset(inputs) == multiset(sorted)
{
  sorted := [];
  var rest := inputs;

  while |rest| > 0
    invariant multiset(inputs) == multiset(sorted) + multiset(rest)
    invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] >= sorted[j]

```

```

invariant forall i, j :: 0 <= i < |sorted| && 0 <= j < |rest| ==> sorted[i] >= rest[j]
decreases |rest|
{
  var oldSorted := sorted;
  var oldRest := rest;
  assert |oldRest| > 0;
  assert forall i, j :: 0 <= i < j < |oldSorted| ==> oldSorted[i] >= oldSorted[j];
  assert forall i, j :: 0 <= i < |oldSorted| && 0 <= j < |oldRest| ==> oldSorted[i] >=
    ↪ oldRest[j];

  var maxIdx := 0;
  var k := 1;
  while k < |oldRest|
    invariant 1 <= k <= |oldRest|
    invariant 0 <= maxIdx < k
    invariant forall t :: 0 <= t < k ==> oldRest[maxIdx] >= oldRest[t]
    decreases |oldRest| - k
    {
      var kk := k;
      var prevMaxIdx := maxIdx;
      if oldRest[kk] > oldRest[maxIdx] {
        maxIdx := kk;
        forall t | 0 <= t < kk
          ensures oldRest[maxIdx] >= oldRest[t]
          {
            assert oldRest[prevMaxIdx] >= oldRest[t];
            assert oldRest[maxIdx] > oldRest[prevMaxIdx];
          }
        assert oldRest[maxIdx] >= oldRest[kk];
      } else {
        forall t | 0 <= t < kk
          ensures oldRest[maxIdx] >= oldRest[t]
          {
            }
        assert oldRest[maxIdx] >= oldRest[kk];
      }
      forall t | 0 <= t < kk + 1
        ensures oldRest[maxIdx] >= oldRest[t]
      {
        if t < kk {
          assert oldRest[maxIdx] >= oldRest[t];
        } else {
          assert t == kk;
          assert oldRest[maxIdx] >= oldRest[kk];
        }
      }
      k := kk + 1;
    }

  assert k == |oldRest|;
  assert forall t :: 0 <= t < |oldRest| ==> oldRest[maxIdx] >= oldRest[t];

```

```

var x := oldRest[maxIdx];
var prefix := oldRest[..maxIdx];
var suffix := oldRest[maxIdx + 1..];
var newRest := prefix + suffix;

assert |prefix| == maxIdx;
assert |suffix| == |oldRest| - (maxIdx + 1);
assert oldRest == prefix + [x] + suffix;
assert newRest == prefix + suffix;
calc {
  multiset(oldRest);
==
  multiset(prefix + [x] + suffix);
==
  multiset(prefix + [x]) + multiset(suffix);
==
  multiset(prefix) + multiset([x]) + multiset(suffix);
==
  multiset(prefix) + multiset(suffix) + multiset([x]);
==
  multiset(prefix + suffix) + multiset([x]);
==
  multiset(newRest) + multiset([x]);
}

sorted := oldSorted + [x];
rest := newRest;

assert |sorted| == |oldSorted| + 1;
assert |rest| == |oldRest| - 1;

forall i, j | 0 <= i < j < |sorted|
  ensures sorted[i] >= sorted[j]
{
  if j < |oldSorted| {
    assert i < |oldSorted|;
    assert sorted[i] == oldSorted[i];
    assert sorted[j] == oldSorted[j];
    assert oldSorted[i] >= oldSorted[j];
  } else {
    assert j == |oldSorted|;
    assert i < |oldSorted|;
    assert sorted[i] == oldSorted[i];
    assert sorted[j] == x;
    assert oldSorted[i] >= oldRest[maxIdx];
  }
}

forall i, j | 0 <= i < |sorted| && 0 <= j < |rest|
  ensures sorted[i] >= rest[j]

```

```

{
  var oldIndex: int;
  if j < maxIdx {
    oldIndex := j;
    assert j < |prefix|;
    assert rest[j] == prefix[j];
    assert prefix[j] == oldRest[j];
    assert rest[j] == oldRest[oldIndex];
  } else {
    oldIndex := j + 1;
    assert maxIdx <= j;
    assert j + 1 < |oldRest|;
    assert 0 <= j - maxIdx;
    assert j - maxIdx < |suffix|;
    assert rest[j] == suffix[j - maxIdx];
    assert suffix[j - maxIdx] == oldRest[maxIdx + 1 + (j - maxIdx)];
    assert maxIdx + 1 + (j - maxIdx) == j + 1;
    assert rest[j] == oldRest[oldIndex];
  }
  assert 0 <= oldIndex < |oldRest|;
  if i < |oldSorted| {
    assert sorted[i] == oldSorted[i];
    assert oldSorted[i] >= oldRest[oldIndex];
  } else {
    assert i == |oldSorted|;
    assert sorted[i] == x;
    assert x >= oldRest[oldIndex];
  }
}

assert multiset(sorted) == multiset(oldSorted) + multiset([x]);
calc {
  multiset(sorted) + multiset(rest);
==
  (multiset(oldSorted) + multiset([x])) + multiset(newRest);
==
  multiset(oldSorted) + (multiset(newRest) + multiset([x]));
==
  multiset(oldSorted) + multiset(oldRest);
==
  multiset(inputs);
}
assert multiset(inputs) == multiset(sorted) + multiset(rest);
}

assert rest == [];
assert multiset(rest) == multiset([]);
assert multiset(sorted) + multiset(rest) == multiset(sorted);
assert multiset(inputs) == multiset(sorted);
}

```


Total Token Usage

Input tokens: 308

Output tokens: 7362

Reasoning tokens: 5725

Sum of 'total tokens': 7670

Experiment Timings

Overall Experiment started at 1780668800185, ended at 1780668953966, lasting 153781ms (153.78 seconds)

Iteration #1 started at 1780668800196, ended at 1780668953966, lasting 153770ms (153.77 seconds)